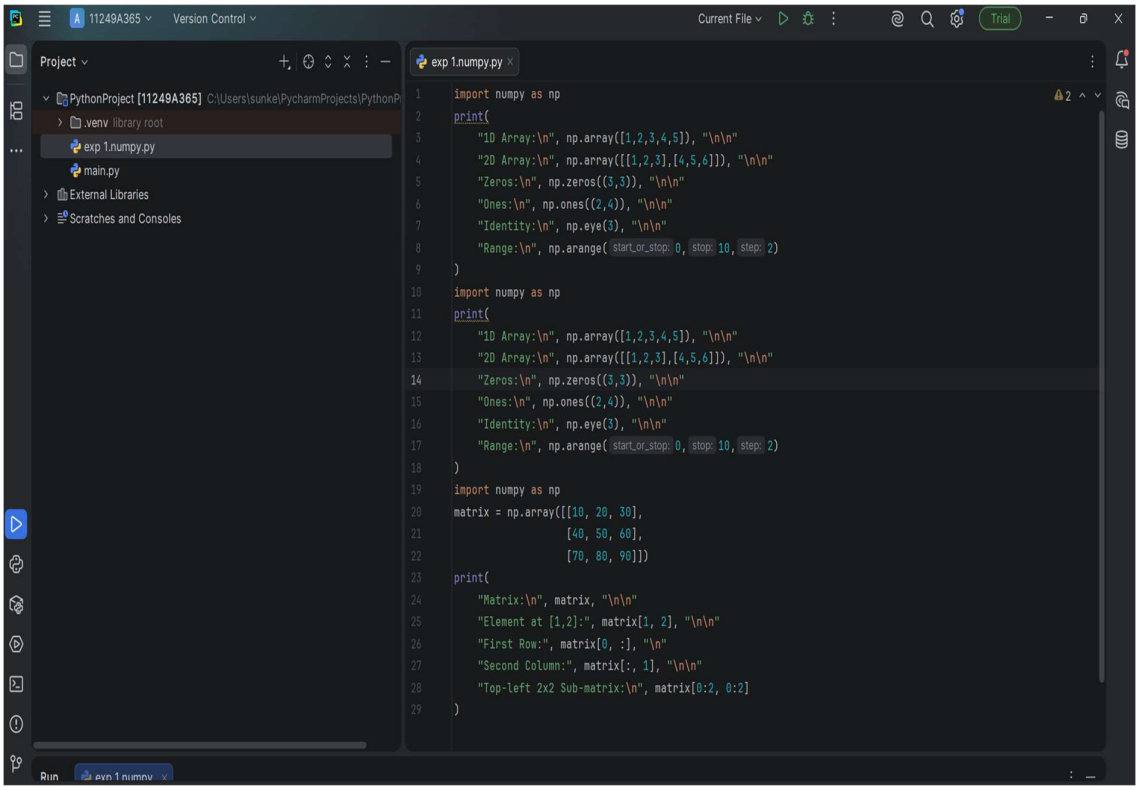
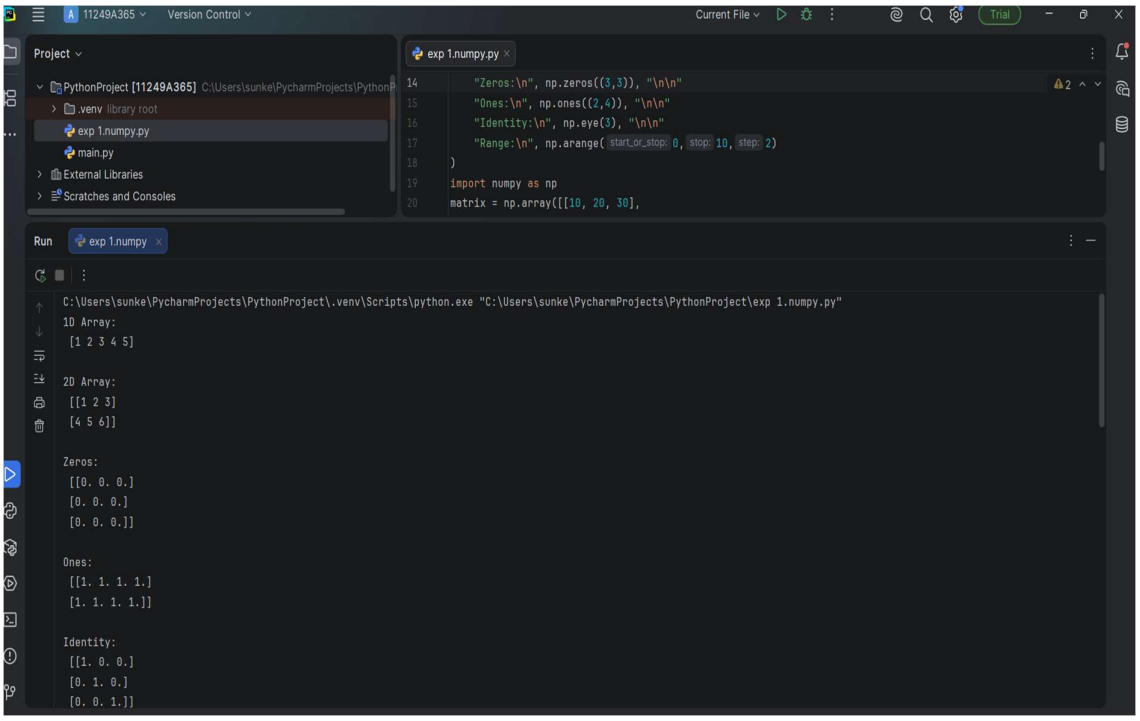
Exp no.1)Introduction to python Libraries for data science

Aim: To learn basic Python libraries for Data Science (NumPy, Pandas, Matplotlib, Scikit learn).

1.1NUMPY(Numerical python)



OUTPUT:


11249A365
Version Control
Current File
Trial
Project
PythonProject [11249A365] C:\Users\sunke\PycharmProjects\PythonP
.venv library root
exp 1.numpy.py
main.py
External Libraries
Scratches and Consoles
exp 1.numpy.py
"Zeros:\n", np.zeros((3,3)), "\n\n"
"Ones:\n", np.ones((2,4)), "\n\n"
"Identity:\n", np.eye(3), "\n\n"
"Range:\n", np.arange(start_or_stop: 0, stop: 10, step: 2)
)
import numpy as np
matrix = np.array([[10, 20, 30],
Run
exp 1.numpy
C:\Users\sunke\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users\sunke\PycharmProjects\PythonProject\exp 1.numpy.py"
1D Array:
[1 2 3 4 5]
2D Array:
[[1 2 3]
[4 5 6]]
Zeros:
[[0. 0. 0.]
[0. 0. 0.]
[0. 0. 0.]]
Ones:
[[1. 1. 1. 1.]
[1. 1. 1. 1.]]
Identity:
[[1. 0. 0.]
[0. 1. 0.]
[0. 0. 1.]]

```python
    "Zeros:\n", np.zeros((3,3)), "\n\n"
    "Ones:\n", np.ones((2,4)), "\n\n"
```

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

Identity:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Range:
 [0 2 4 6 8]
Matrix:
 [[10 20 30]
 [40 50 60]
 [70 80 90]]

Element at [1,2]: 60

First Row: [10 20 30]
Second Column: [20 50 80]

Top-left 2x2 Sub-matrix:
 [[10 20]
 [40 50]]

Process finished with exit code 0
```

```python
    "Zeros:\n", np.zeros((3,3)), "\n\n"
    "Ones:\n", np.ones((2,4)), "\n\n"
    "Identity:\n", np.eye(3), "\n\n"
    "Range:\n", np.arange(start_or_stop: 0, stop: 10, step: 2)
)
import numpy as np
matrix = np.array([[10, 20, 30],
```

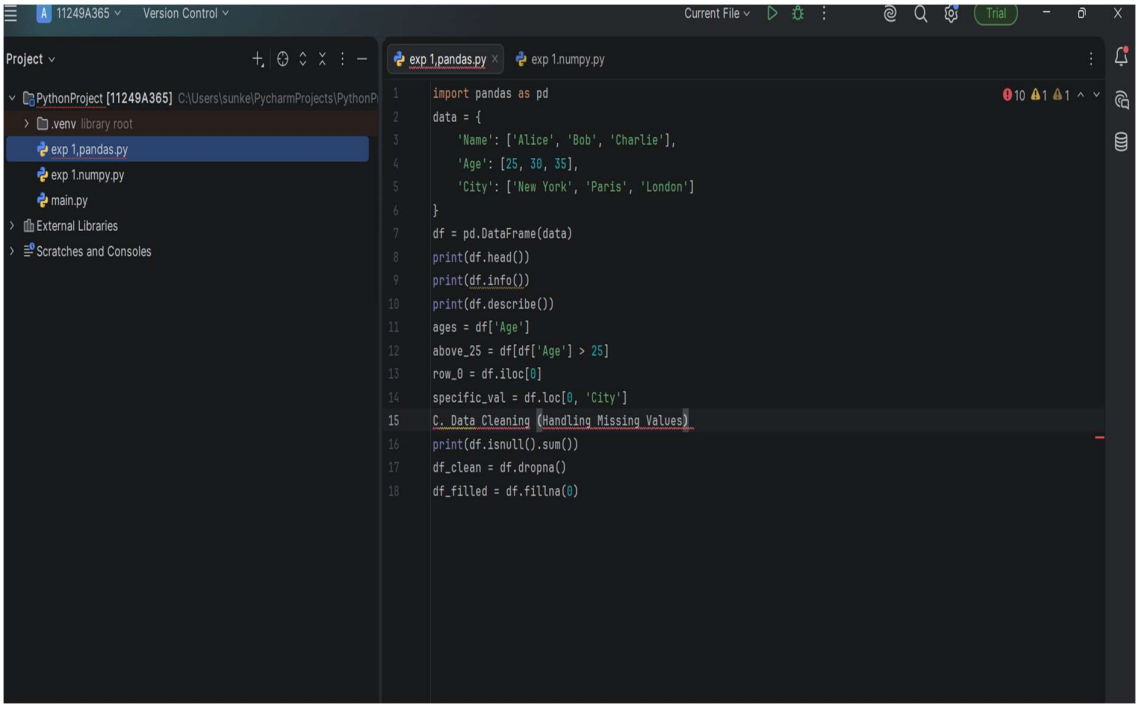
```
Range:
 [0 2 4 6 8]
1D Array:
 [1 2 3 4 5]

2D Array:
 [[1 2 3]
 [4 5 6]]

Zeros:
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

Ones:
 [[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

1.2)Pandas (python data analysis Library)



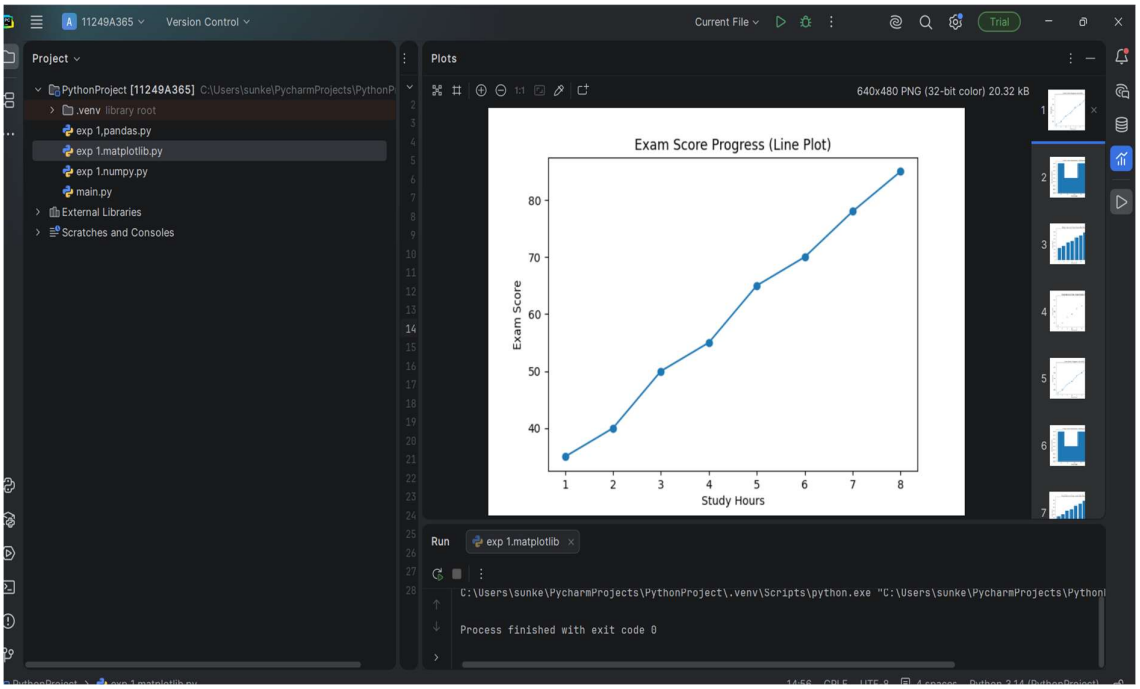
OUTPUT:

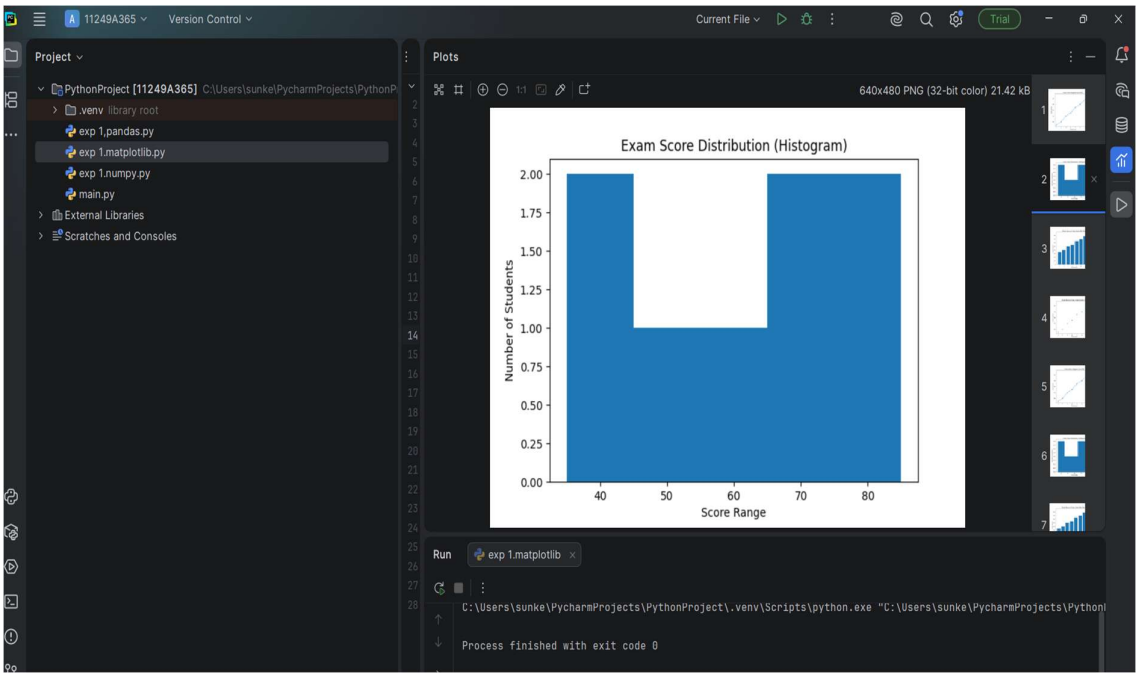
```
C:\Users\sunke\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users\sunke\PycharmProjects\PythonProject\exp 1,pandas.py"
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London
<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      str
 1   Age     3 non-null      int64
 2   City    3 non-null      str
dtypes: int64(1), str(2)
memory usage: 204.0 bytes
None
        Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
Name    0
Age     0
City    0
dtype: int64
```

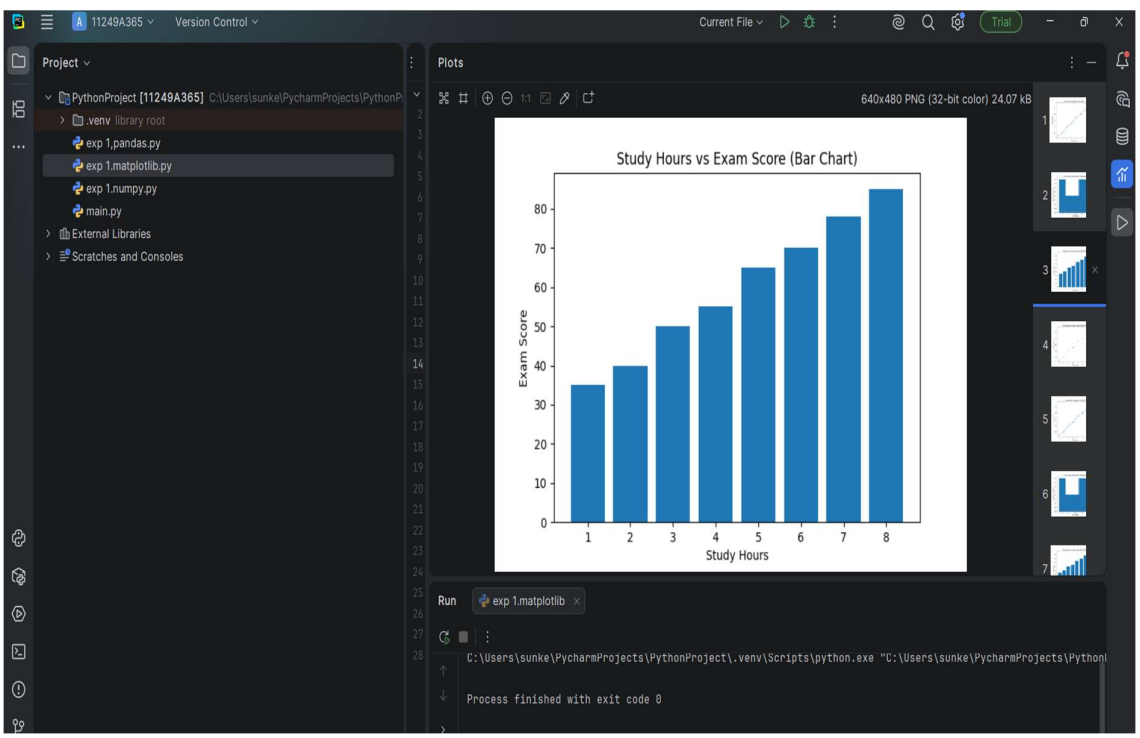
1.3)Matplotlib(plotting Library)

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data = {
    'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8],
    'ExamScore': [35, 40, 50, 55, 65, 70, 78, 85]
}
df = pd.DataFrame(data)
plt.scatter(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Scatter Plot)")
plt.show()
plt.plot(df['StudyHours'], df['ExamScore'], marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress (Line Plot)")
plt.show()
plt.hist(df['ExamScore'], bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution (Histogram)")
plt.show()
plt.bar(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Bar Chart)")
plt.show()
```

OUTPUT:



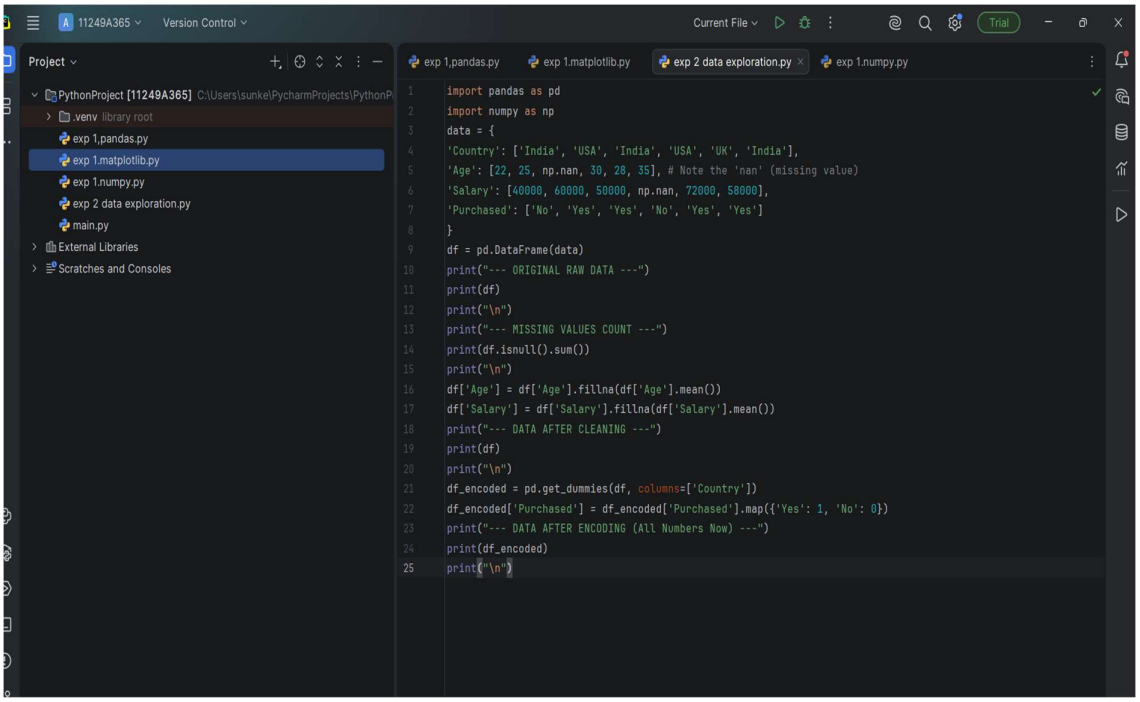




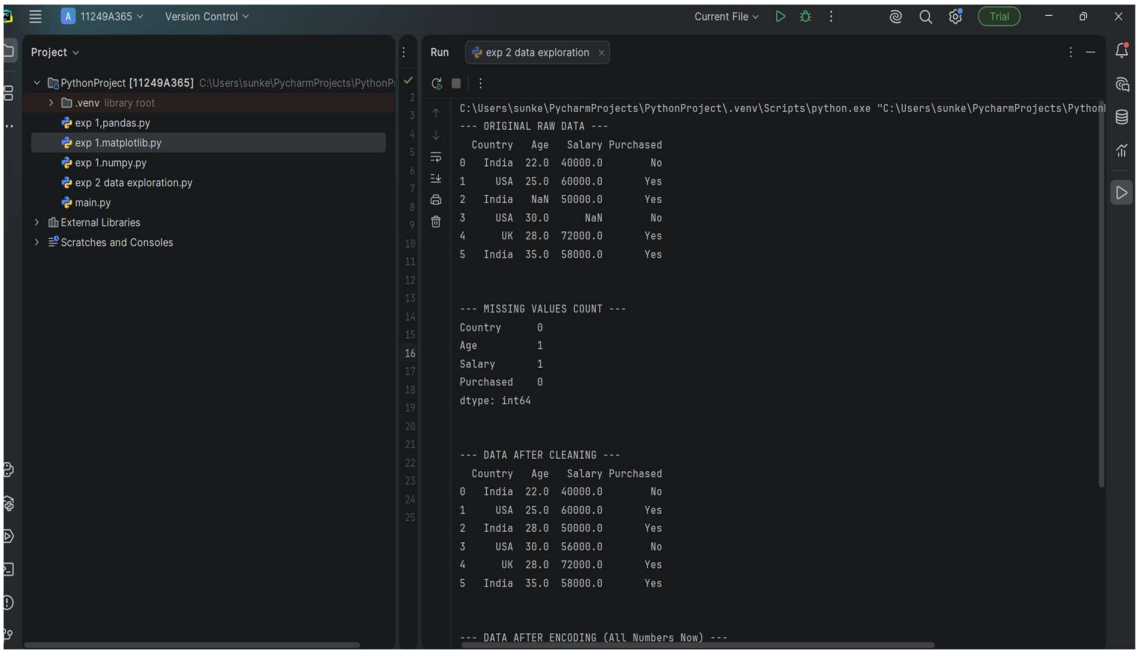
Result: Basic operations using these libraries were performed successfully

EXP NO.2.) Perform Data exploration and preprocessing in python

Aim: To perform data exploration and preprocessing in Python.



OUTPUT:



Result: Data was explored and preprocessed successfully using Python.

Ex No:03)

Implement regularised Linear regression

Aim: To implement regularized linear regression to improve model performance and control overfitting.

Program:

```python
from sklearn.model_selection import train_test_split
diabetes = datasets.load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: diabetes.data, diabetes.target, test_size=0.2, random_state=42
)
lr = LinearRegression().fit(X_train, y_train)
ridge = Ridge(alpha=1.0).fit(X_train, y_train)
lasso = Lasso(alpha=0.1).fit(X_train, y_train)
print(f"Linear Regression MSE: {mean_squared_error(y_test, lr.predict(X_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test, ridge.predict(X_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test, lasso.predict(X_test)):.2f}")
```

OUTPUT:

```
C:\Users\Administrator\PycharmProjects\dstlab_11249A365\.venv\Scripts\python.exe C:\Users\Administrator\PycharmProjects\dstlab_11249A365\exp3.py
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 2798.19

Process finished with exit code 0
```

Result: The model was successfully implemented, and regularization helped reduce overfitting and improved prediction accuracy

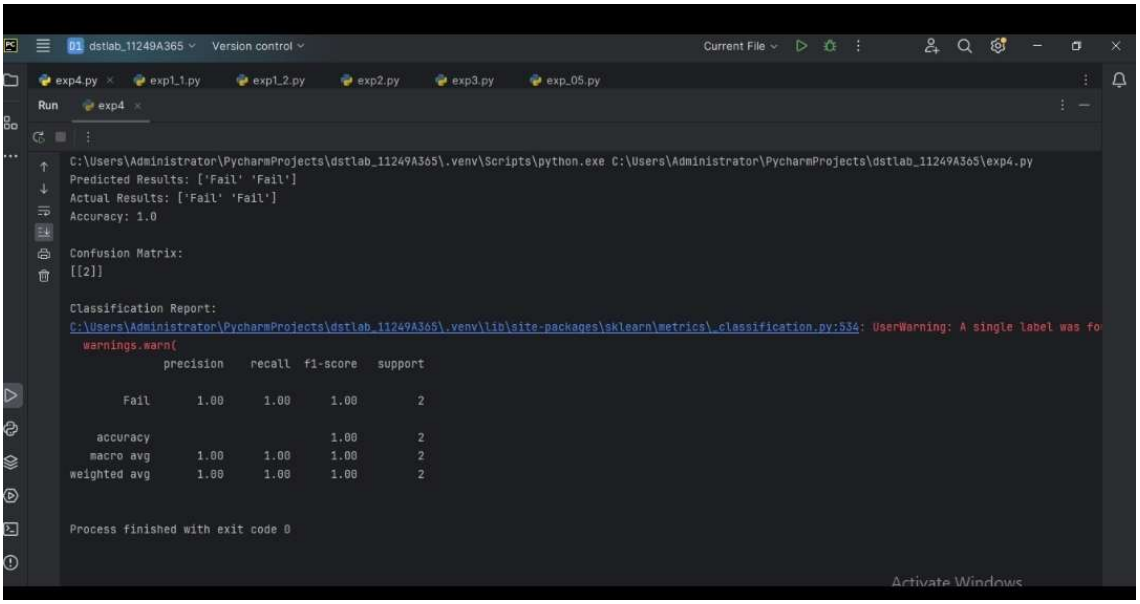
Ex No:04)Implement Naive Bayes classifier for dataset stored as CSV file

Aim: To implement the Naive Bayes classifier using a dataset stored in a CSV file for classification.

Program:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
data = {
    'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
}
df = pd.DataFrame(data)
X = df[['StudyHours', 'Attendance']]
y = df['Result']
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.25, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Results:", y_pred)
print("Actual Results:", y_test.values)
print("Accuracy:", accuracy)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
```

OUTPUT:



```
C:\Users\Administrator\PycharmProjects\dstlab_11249A365\.venv\Scripts\python.exe C:\Users\Administrator\PycharmProjects\dstlab_11249A365\exp4.py
Predicted Results: ['Fail' 'Fail']
Actual Results: ['Fail' 'Fail']
Accuracy: 1.0

Confusion Matrix:
[[2]]

Classification Report:
C:\Users\Administrator\PycharmProjects\dstlab_11249A365\.venv\lib\site-packages\sklearn\metrics\_classification.py:534: UserWarning: A single label was fo
  warnings.warn(
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2


Process finished with exit code 0
```

Result:

The Naive Bayes model was successfully implemented, and it effectively classified the data with good accuracy.

Ex No:05)Implement regularized logistic regression

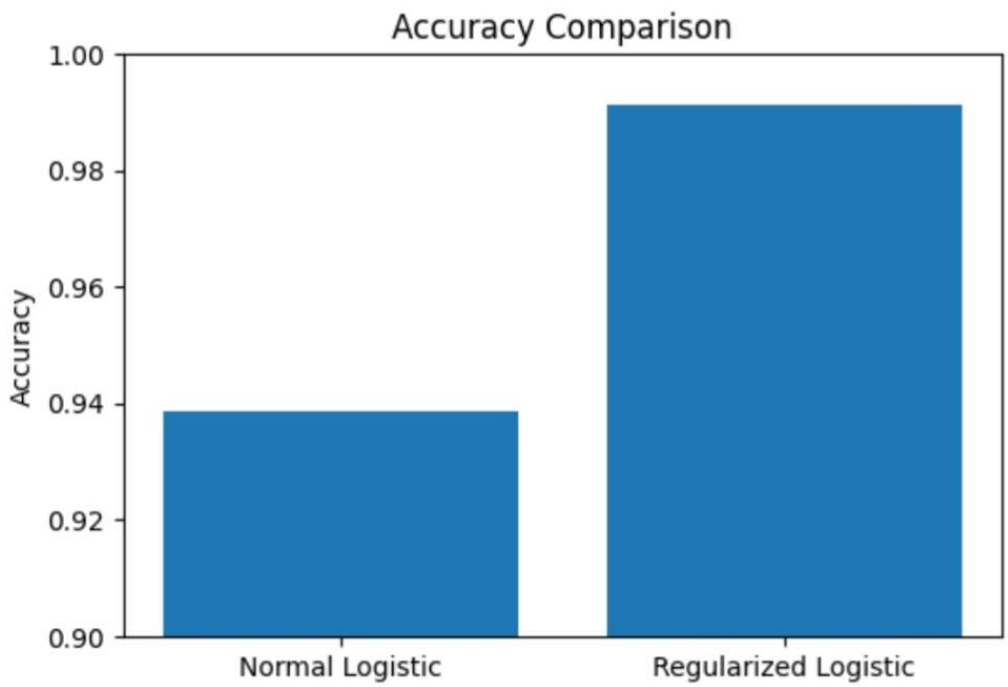
Aim: To implement regularized logistic regression for classification while reducing overfitting.

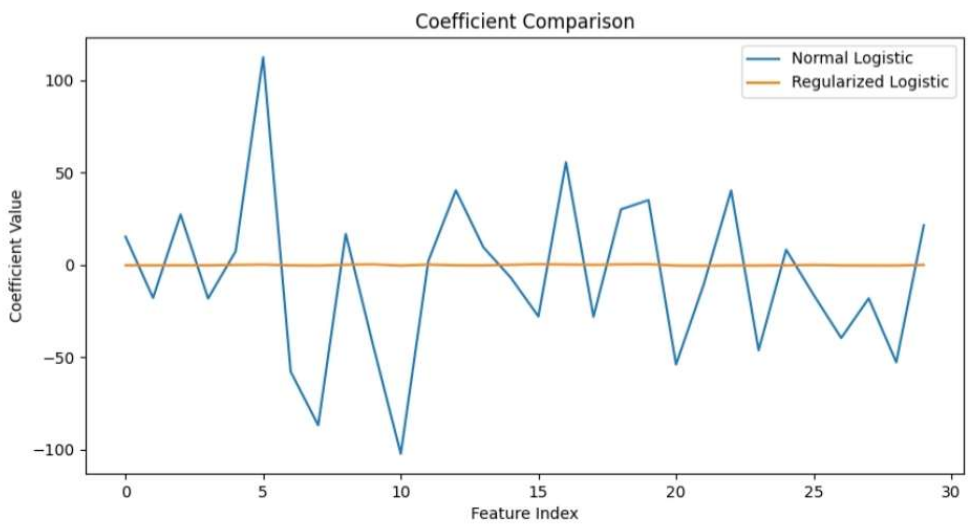
Program:

```python
import ...
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
normal_model = LogisticRegression(
    C=1e6, penalty='l2', solver='liblinear'
)
normal_model.fit(X_train_scaled, y_train)
normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)
regularized_model = LogisticRegression(
    C=0.1, penalty='l2', solver='liblinear'
)
regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)
plt.figure(figsize=(10, 5))
```

```python
plt.plot( *args: normal_model.coef_.flatten(), label='Normal Logistic')
plt.plot( *args: regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
plt.figure(figsize=(6, 4))
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]
plt.bar(models, accuracies)
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim( *args: 0.9, 1.0)
plt.show()
```

OUTPUT:





Result:

The model was successfully implemented, and regularization improved accuracy and prevented overfitting.

Ex No:06)Build models using different Ensembling techniques

Aim: To build models using different ensembling techniques to improve prediction performance

PROGRAM:

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: iris.data, iris.target, test_size=0.2, random_state=42
)
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))
]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
).fit(X_train, y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

OUTPUT:

Result:

The ensemble models were successfully implemented, and they provided better accuracy compared to individual models.

Ex No:07)Build models using Decision trees

Aim: To build a model using Decision Trees for classification or prediction.

Program:

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: iris.data, iris.target, test_size=0.2, random_state=42
)
dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, dt_clf.predict(X_test)):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

OUTPUT:

```
C:\Users\Administrator\PycharmProjects\dstlab_11249A365\.venv\Scripts\python.exe C:\Users\Administrator\PycharmProjects\dstlab_11249A365\exp7.py
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.        0.        0.39426873 0.60573127]

Process finished with exit code 0
```

Result:

The Decision Tree model was successfully implemented and provided good prediction performance.

Ex No:08)Build model using SVM with different kernels

Aim: To build an SVM model using different kernel functions for classification**.**

Program:

```python
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.2)
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)
```

OUTPUT:

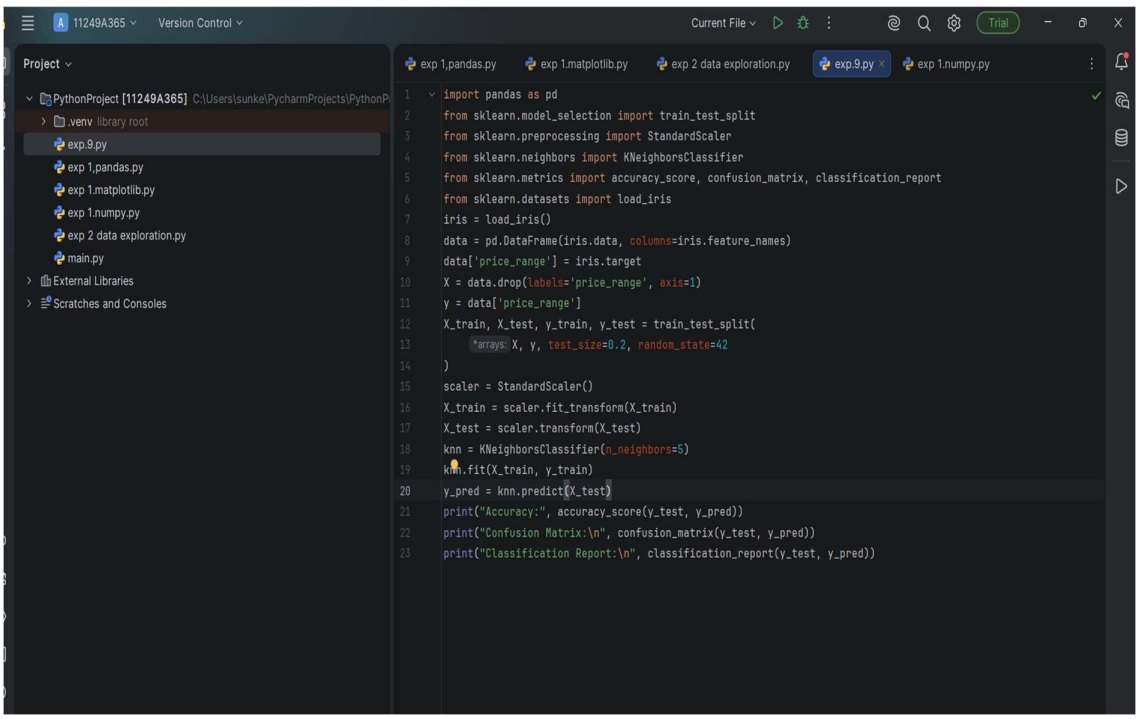
Result:

The SVM models were successfully implemented with various kernels, and they achieved good classification performance.

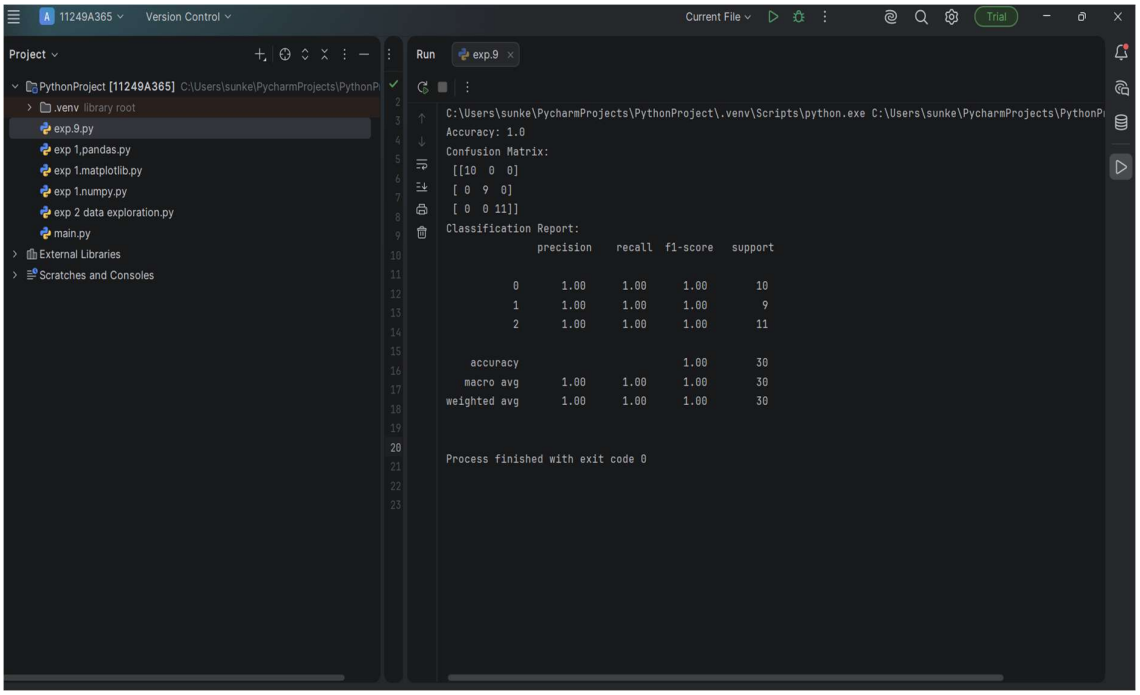
Ex No:09)Implement K-NN algorithm to classify a dataset

Aim: To implement the K-Nearest Neighbors (K-NN) algorithm to classify a dataset

Program:



OUTPUT:



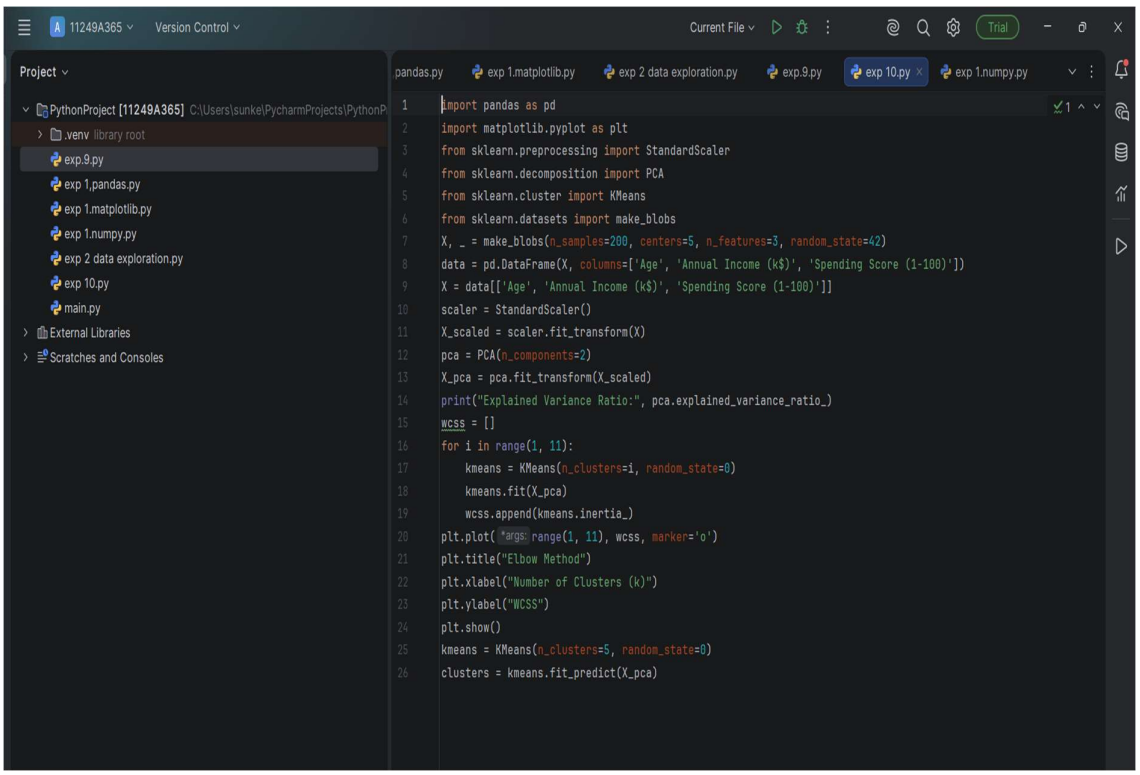
Result:

The K-NN model was successfully implemented and it classified the dataset with good accuracy

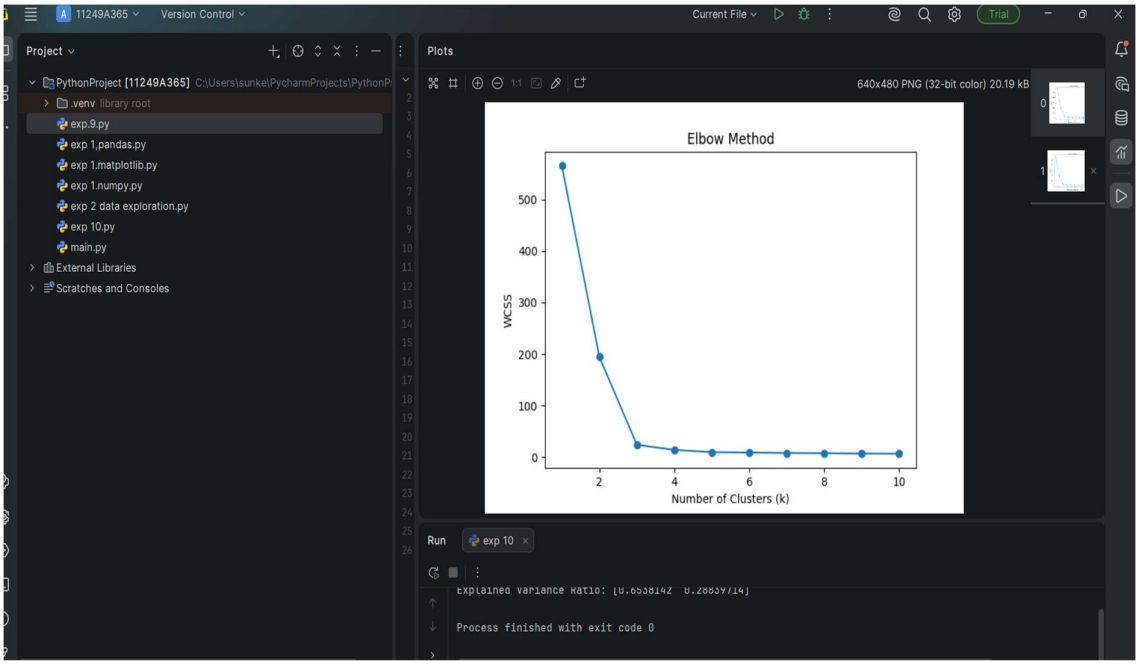
Ex No:10)Build model to perform Clustering using K-means after applying PCA and determining

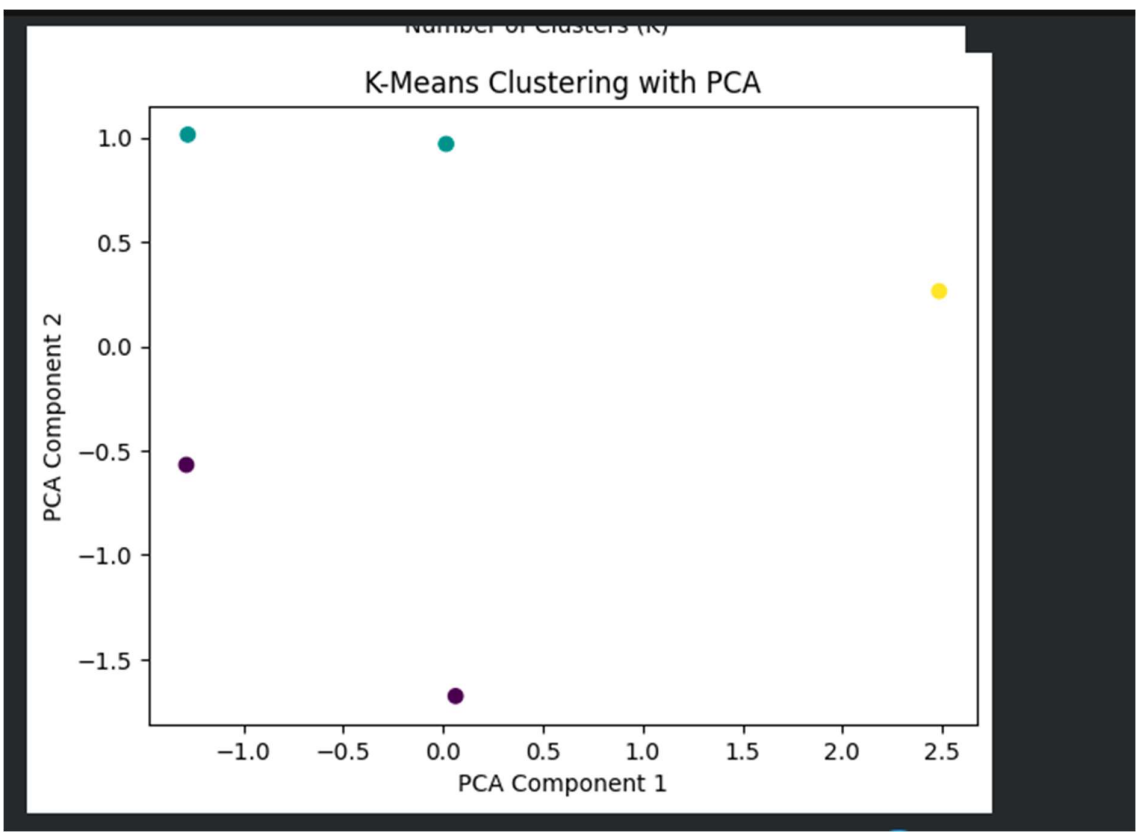
Aim: To build a clustering model using K-Means after applying PCA for dimensionality reduction and determining optimal clusters.

Program:



OUTPUT:





Result:

The K-Means clustering model was successfully implemented after PCA, and the data was grouped effectively into clusters